



**UNIVERSIDAD
TÉCNICA DE
MANABÍ**

**FACULTAD DE CIENCIAS INFORMÁTICAS
CARRERA DE TECNOLOGIAS DE LA INFORMACION**

**ASIGNATURA
DESARROLLO DE SISTEMAS INFORMATICOS**

UNIDAD 1:
Desarrollo de software Introducción y
Paradigmas de diseño

TAREA:
Estudio de caso - Sistema de Gestión de Incidentes (Help Desk)

INTEGRANTES:
CARLOS ROBERTO MANCERO CAIMITO

DOCENTE:
Ing. Jorge Arun Zambrano Cedeño, Mg.

**PORTOVIEJO - ECUADOR
MAYO, 2026**



Actividades a desarrollar en la Unidad 1

Actividad # 1

Objetivos de la tarea

Diseñar la arquitectura lógica de un sistema de gestión de incidentes técnicos (Help Desk) para infraestructura de servidores que permita aplicar correctamente patrones de diseño de software y flujos de trabajo profesionales (Gitflow), fortaleciendo la capacidad de análisis, abstracción y buenas prácticas en el desarrollo de soluciones escalables, reutilizables y mantenibles.

Descripción de la Actividad

El estudiante deberá diseñar un sistema de Help Desk orientado a la gestión de fallos en un Data Center. El sistema debe permitir reportar incidentes (hardware, red, software), asignar técnicos especializados y gestionar estados de resolución. Para lograr una arquitectura clara y profesional, se deberá utilizar obligatoriamente un repositorio en GitHub bajo el modelo Gitflow y aplicar al menos tres patrones de diseño de software, justificando su elección y uso técnico.

Instrucciones

1. Análisis del problema

- Identificar los actores principales (Usuario final, Técnico de soporte, Administrador de sistemas).
- Definir los requerimientos funcionales (Crear ticket, categorizar fallo, asignar técnico) y no funcionales (Escalabilidad, mantenibilidad, trazabilidad de versiones).

2. Diseño del sistema

- Crear un repositorio público en GitHub configurando la estructura de ramas Gitflow (master, develop y ramas feature/ para avances).
- Crear un diagrama de casos de uso para entender las interacciones del personal de TI con la infraestructura.



- Realizar un diagrama de clases UML donde se evidencien los objetos y su relación bajo principios SOLID.
- Diseñar la arquitectura general del sistema integrando el flujo de versiones.

3. Aplicación de patrones de diseño

Utilizar al menos tres patrones. Algunos recomendados:

- Singleton: Para la instancia central del motor de gestión de tickets y conexión a datos.
- Observer: Indispensable para el sistema de notificaciones automáticas a técnicos ante nuevos incidentes (Base de la programación reactiva).
- Factory Method: Para la creación de diferentes tipos de incidentes según el área (Red, Hardware, Software).
- MVC (Model-View-Controller): Como patrón estructural para organizar la lógica de negocio y la interfaz.

4. Documentación del diseño

- Explicar el propósito de cada patrón utilizado y cómo resuelve un problema específico del Help Desk.
- Justificar el uso de Gitflow: Incluir el enlace al repositorio de GitHub y capturas de pantalla del historial de ramas/commits que evidencien el flujo de trabajo.
- Incluir diagramas de clases y secuencia donde se refleje la implementación de los patrones.



1. INTRODUCCIÓN

Actualmente, los Data Centers representan uno de los componentes más importantes dentro de la infraestructura tecnológica de las organizaciones, debido a que permiten almacenar, procesar y administrar grandes cantidades de información y servicios críticos, como consecuencia de ello, la correcta gestión de incidentes relacionados con hardware, red y software se vuelve indispensable para garantizar la continuidad operativa y minimizar tiempos de inactividad.

En este contexto, los sistemas Help Desk constituyen una herramienta fundamental para administrar incidencias técnicas, registrar solicitudes, asignar personal especializado y realizar seguimiento al estado de resolución de cada problema reportado.

El presente trabajo tiene como finalidad diseñar la arquitectura lógica de un sistema Help Desk orientado a la gestión de incidentes en un Data Center, aplicando metodologías profesionales de desarrollo de software mediante el uso de UML, patrones de diseño y control de versiones con Gitflow.

Adicionalmente, se emplean patrones de diseño reconocidos como Singleton, Observer, Factory Method y MVC, permitiendo construir una solución escalable, reutilizable, organizada y mantenible bajo principios modernos de ingeniería de software.



2. OBJETIVOS

2.1 Objetivo General

Diseñar un sistema Help Desk para la gestión de incidentes en un Data Center aplicando diagramas UML, patrones de diseño de software y el modelo de control de versiones Gitflow, con el propósito de desarrollar una arquitectura organizada, escalable y mantenible.

2.2 Objetivos Específicos

- Analizar los requerimientos funcionales y no funcionales del sistema.
- Identificar los actores involucrados en el proceso de soporte técnico.
- Diseñar diagramas UML que representen la estructura y comportamiento del sistema.
- Aplicar patrones de diseño para optimizar la arquitectura de software.
- Implementar el flujo de trabajo Gitflow utilizando un repositorio GitHub.
- Documentar técnicamente el diseño del sistema siguiendo normas de presentación universitaria.

3. ANALISIS DEL PROBLEMA

Los Data Centers manejan múltiples servicios tecnológicos críticos para las organizaciones, por lo que cualquier incidente relacionado con infraestructura tecnológica puede afectar considerablemente la disponibilidad de los servicios.

Actualmente, muchos procesos de soporte técnico se realizan manualmente, dificultando el seguimiento de incidencias, la asignación adecuada de técnicos y el control de estados de resolución.

Entre los principales problemas identificados se encuentran:

- Falta de control centralizado de incidentes.
- Dificultad para clasificar fallos según el área afectada.



- Escasa trazabilidad de tickets.
- Ausencia de notificaciones automáticas.
- Poca organización en el seguimiento técnico.
- Deficiente control de versiones durante el desarrollo del software.

Como solución se plantea el diseño de un sistema Help Desk que permita administrar incidentes técnicos de manera eficiente mediante una arquitectura basada en patrones de diseño y control de versiones profesional.

4. ACTORES PRINCIPALES DEL SISTEMA

4.1 Usuario Final

Representa al personal que reporta incidentes dentro del Data Center.

Funciones principales:

- Crear tickets.
- Consultar estados.
- Adjuntar evidencias.
- Dar seguimiento a incidencias.

4.2 Técnico de Soporte

Encargado de resolver los incidentes registrados en el sistema.

Funciones principales:

- Ver tickets asignados.
- Actualizar estados.
- Resolver incidentes.
- Registrar soluciones técnicas.

4.3 Administrador del Sistema

Responsable de administrar el sistema y coordinar las actividades de soporte.

Funciones principales:

- Asignar técnicos.
- Gestionar usuarios.



- Generar reportes.
- Supervisar el flujo de incidencias.

5. REQUERIMIENTO DEL SISTEMA

5.1 Requerimientos Funcionales

El sistema deberá permitir:

- Crear tickets de incidentes.
- Clasificar incidentes por categoría.
- Asignar técnicos especializados.
- Actualizar estados de resolución.
- Registrar soluciones.
- Adjuntar evidencias.
- Enviar notificaciones automáticas.
- Generar reportes administrativos.
- Gestionar usuarios y permisos.

5.2 Requerimientos No Funcionales

El sistema deberá cumplir con:

- Escalabilidad.
- Mantenibilidad.
- Disponibilidad.
- Seguridad.
- Reutilización de componentes.
- Trazabilidad de versiones mediante Gitflow.
- Arquitectura organizada y modular.



6. DISEÑO DEL SISTEMA

El sistema Help Desk fue diseñado siguiendo una arquitectura modular orientada a objetos, permitiendo separar claramente las responsabilidades de cada componente.

Para garantizar una adecuada administración del desarrollo se implementó Gitflow como metodología de control de versiones utilizando GitHub.

La estructura de ramas utilizada fue:

- main
- develop
- feature/crear-ticket
- feature/gestion-estados
- feature/asignar-tecnico

Esta estructura permitió trabajar de manera organizada y mantener trazabilidad sobre cada avance realizado.

7. DIAGRAMA DE CASO DE USO

El diagrama de casos de uso representa las interacciones entre los actores y el sistema Help Desk.

Funcionalidades principales representadas:

- Crear Ticket
- Consultar Estado Ticket
- Adjuntar Evidencia
- Ver Tickets Asignados
- Actualizar Estado
- Resolver Incidente



- Registrar Solución
- Asignar Técnico
- Gestionar Usuarios
- Generar Reportes

Análisis del Diagrama

El diagrama evidencia la interacción entre los usuarios finales, técnicos y administradores del sistema, permitiendo identificar claramente las responsabilidades y operaciones principales de cada actor.(IMAGEN EN EVIDENCIAS)

8. DIAGRAMA UML DE CLASES

El diagrama UML de clases representa la estructura lógica del sistema, mostrando las clases, atributos, métodos y relaciones entre objetos.

Clases principales

- UsuarioFinal
- TecnicoSoporte
- AdministradorSistema
- Ticket
- Evidencia
- Reporte
- Categoria
- Notificacion
- ConexionBD
- IncidenteFactory



Análisis del Diagrama UML

El diagrama permite observar la relación entre las entidades principales del sistema, aplicando conceptos de orientación a objetos y principios SOLID, asimismo, se evidencian componentes asociados a los patrones Singleton, Observer y Factory Method. (IMAGEN EN EVIDENCIA)

9. ARQUITECTURA GENERAL DEL SISTEMA

La arquitectura del sistema fue diseñada bajo el patrón MVC (Model-View-Controller), permitiendo separar la lógica de negocio, la presentación y el control del sistema.

Estructura General

- Vista (Interfaz de usuario)
- Controladores
- Lógica de negocio
- Persistencia de datos

Componentes Integrados

- Patrón Singleton para la conexión centralizada.
- Patrón Observer para las notificaciones.
- Patrón Factory Method para la creación de incidentes.

Análisis de Arquitectura

La arquitectura implementada permite mantener una adecuada separación de responsabilidades, facilitando la escalabilidad y mantenibilidad del sistema.



10. APLICACIÓN DE PATRONES DE DISEÑO

10.1 Patrón Singleton

Propósito

Garantizar que exista una única instancia de conexión a la base de datos dentro del sistema.

Implementación

Se implementó mediante la clase:

- ConexionBD

Problema que resuelve

Evita la creación innecesaria de múltiples conexiones, optimizando recursos y manteniendo consistencia en el acceso a datos.

Justificación Técnica

El patrón Singleton permite centralizar la administración de recursos críticos del sistema, mejorando el rendimiento y la organización de la arquitectura.

10.2 Patrón Observer

Propósito

Permitir la notificación automática a técnicos cuando se genera un nuevo incidente.

Implementación

Se implementó mediante:

- Ticket
- Notificacion
- TecnicoSoporte



Problema que resuelve

Automatiza el proceso de comunicación entre el sistema y los técnicos encargados.

Justificación Técnica

El patrón Observer facilita la programación reactiva, permitiendo responder automáticamente ante eventos del sistema.

10.3 Patrón Factory Method

Propósito

Crear diferentes tipos de incidentes según la categoría correspondiente.

Implementación

Se implementó mediante:

- IncidenteFactory

Tipos de incidentes

- Hardware
- Red
- Software

Problema que resuelve

Permite desacoplar la creación de objetos de la lógica principal del sistema.

Justificación Técnica

Factory Method facilita la extensibilidad del sistema permitiendo incorporar nuevas categorías sin modificar la lógica existente.



10.4 Patrón MVC (Model-View-Controller)

Propósito

Separar la lógica de negocio, la interfaz y el control de flujo del sistema.

Implementación

- Modelo: Ticket, Usuario, Categoria
- Vista: Interfaces del sistema
- Controlador: Gestión de tickets y procesos

Problema que resuelve

Reduce el acoplamiento entre componentes.

Justificación Técnica

MVC mejora la organización del sistema y facilita futuras modificaciones y mantenimiento.

11. IMPLMENTACION DE GITFLOW

Para garantizar un control de versiones profesional se implementó Gitflow utilizando GitHub.

Estructura de ramas utilizada

- main
- develop
- feature/crear-ticket
- feature/gestion-estados
- feature/asignar-tecnico



Flujo de trabajo aplicado

- Desarrollo en ramas feature.
- Integración en develop.
- Validación y consolidación.
- Publicación final en main.

Beneficios obtenidos

- Mejor organización del proyecto.
- Trazabilidad de cambios.
- Separación de funcionalidades.
- Mayor control del desarrollo.
- Historial claro de commits y merges.

12. DIAGRAMA DE SECUENCIA

El diagrama de secuencia representa el flujo de creación de tickets dentro del sistema.

Flujo representado

1. Usuario crea ticket.
2. Sistema solicita creación del incidente.
3. Factory Method genera el ticket.
4. Sistema envía notificación.
5. Técnico recibe alerta.
6. Técnico actualiza estado.

Análisis del Diagrama

El diagrama evidencia la interacción dinámica entre los componentes del sistema y la implementación práctica de los patrones Observer y Factory Method.



13. PRINCIPIO SOLID APLICADOS

Single Responsibility Principle

Cada clase cumple una única responsabilidad específica.

Ejemplo:

- Ticket administra incidencias.
- Notificacion gestiona alertas.
- ConexionBD administra la conexión.

Open/Closed Principle

El sistema puede extenderse agregando nuevos tipos de incidentes sin modificar el código principal.

Liskov Substitution Principle

Las clases derivadas pueden sustituir correctamente a sus clases base.

Interface Segregation Principle

Las clases utilizan únicamente las interfaces necesarias.

Dependency Inversion Principle

Los módulos dependen de abstracciones y no de implementaciones concretas.

14. RESULTADOS OBTENIDOS

Mediante el desarrollo del presente trabajo se logró:

- Diseñar una arquitectura lógica profesional.
- Aplicar patrones de diseño reconocidos.
- Implementar control de versiones mediante Gitflow.
- Representar gráficamente la estructura del sistema.
- Organizar adecuadamente los componentes del software.



- Mejorar la mantenibilidad y escalabilidad del sistema.

15. CONCLUSIONES

- El diseño del sistema Help Desk permitió comprender la importancia de la arquitectura de software en la gestión de incidentes tecnológicos.
- La aplicación de patrones de diseño mejoró la organización, reutilización y mantenibilidad del sistema.
- Gitflow facilitó el control de versiones y permitió una adecuada administración de ramas y funcionalidades.
- UML permitió representar de forma clara tanto la estructura como el comportamiento del sistema.
- La separación de responsabilidades mediante principios SOLID contribuyó al desarrollo de una solución más profesional y escalable.

16. RECOMENDACIONES

- Implementar futuras mejoras relacionadas con inteligencia artificial para clasificación automática de incidentes.
- Incorporar autenticación avanzada para fortalecer la seguridad.
- Desarrollar dashboards estadísticos para monitoreo en tiempo real.
- Integrar bases de datos reales y servicios de correo electrónico.
- Continuar utilizando metodologías de control de versiones en proyectos futuros.

17. EVIDENCIAS

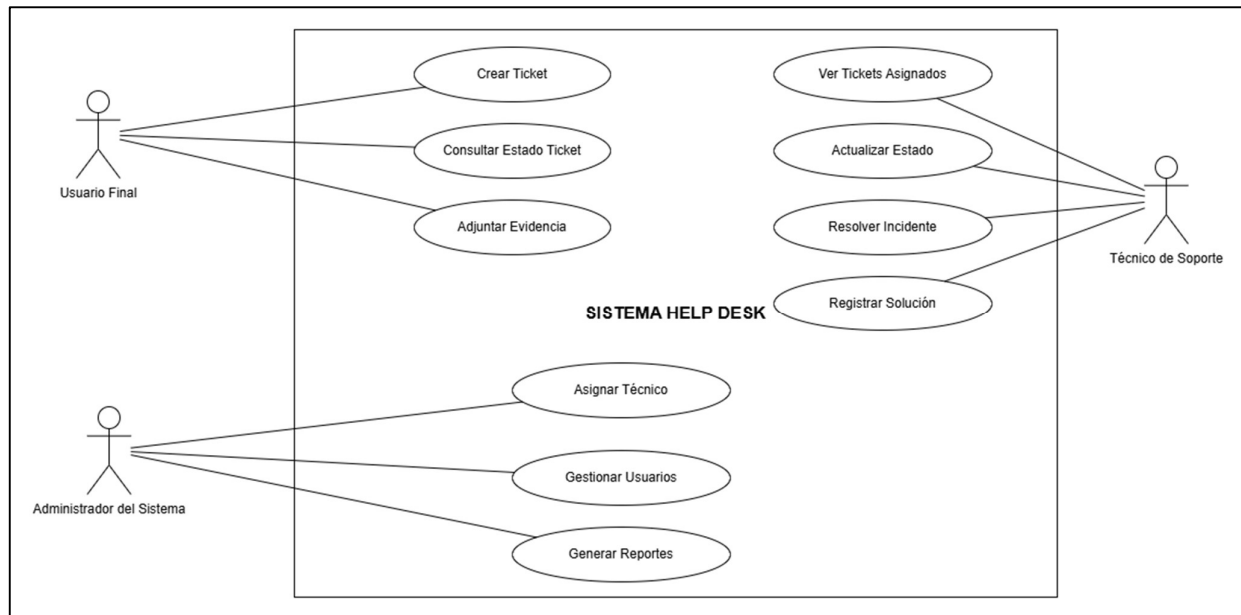


DIAGRAMA DE CASO DE USO

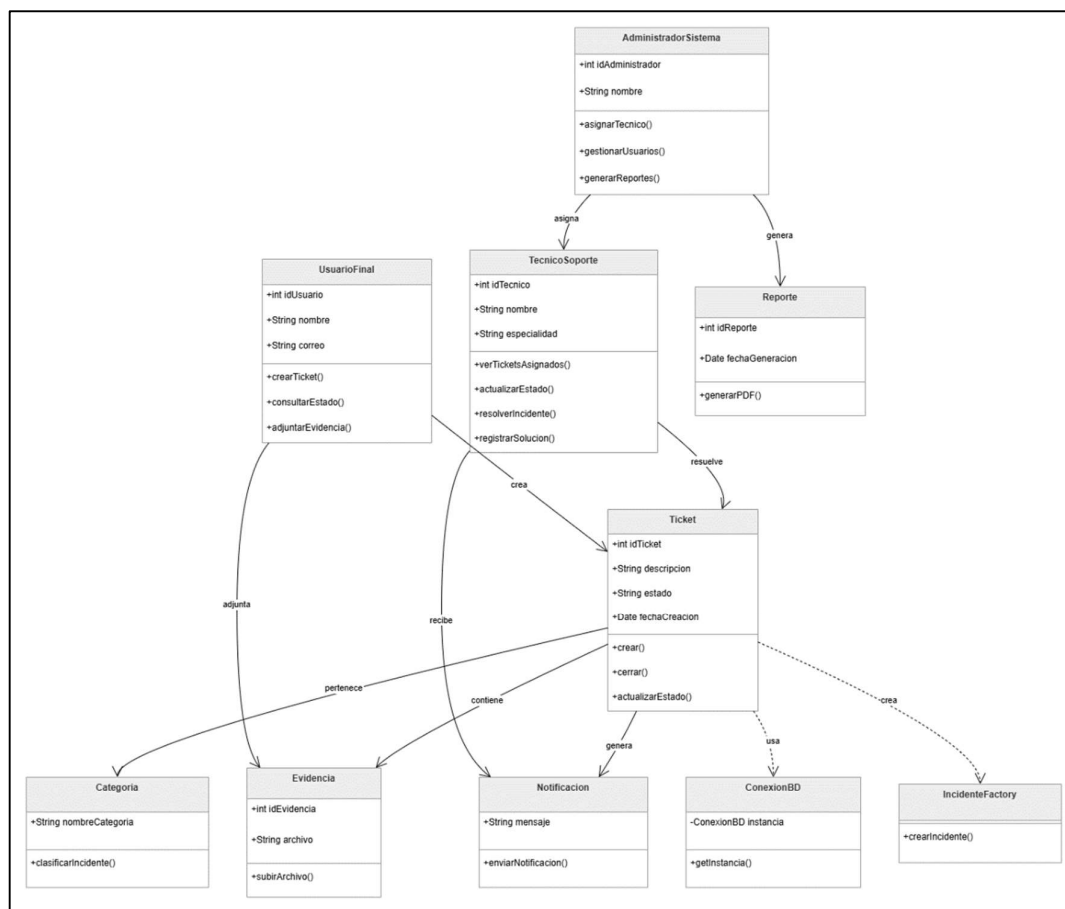


DIAGRAMA UML DE CLASES

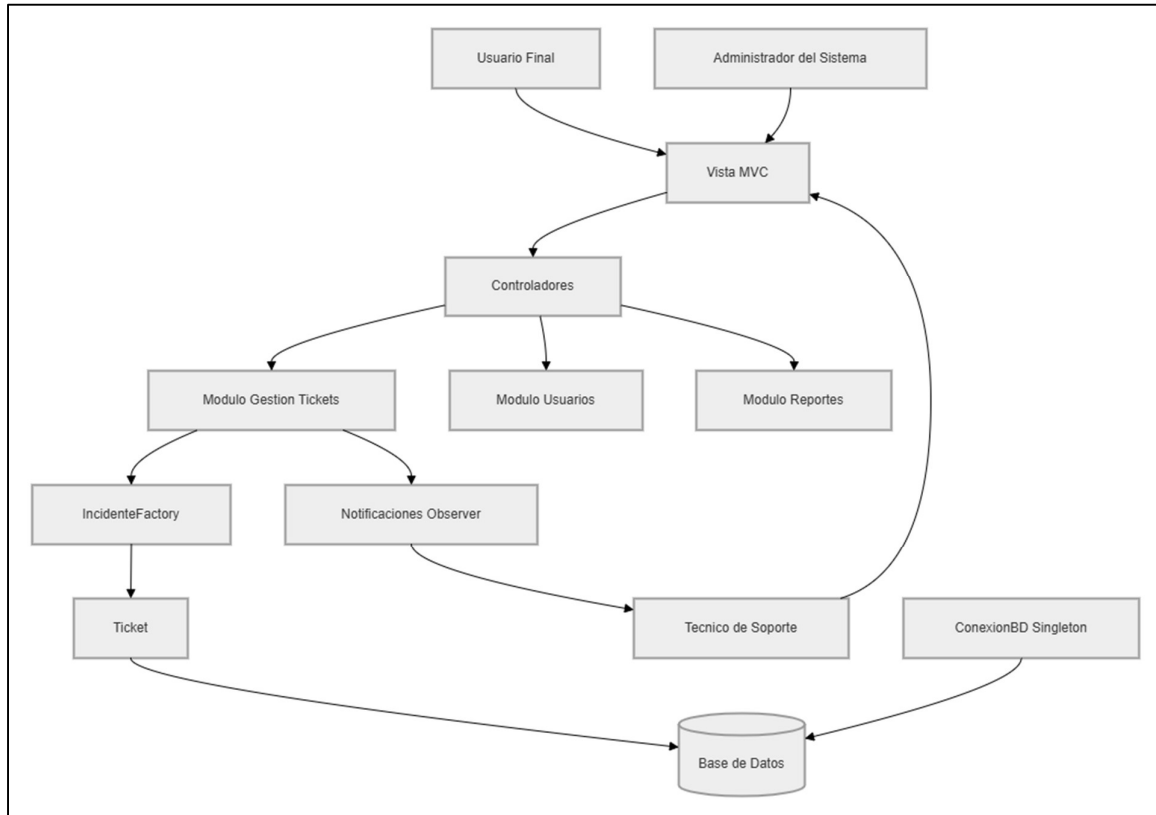
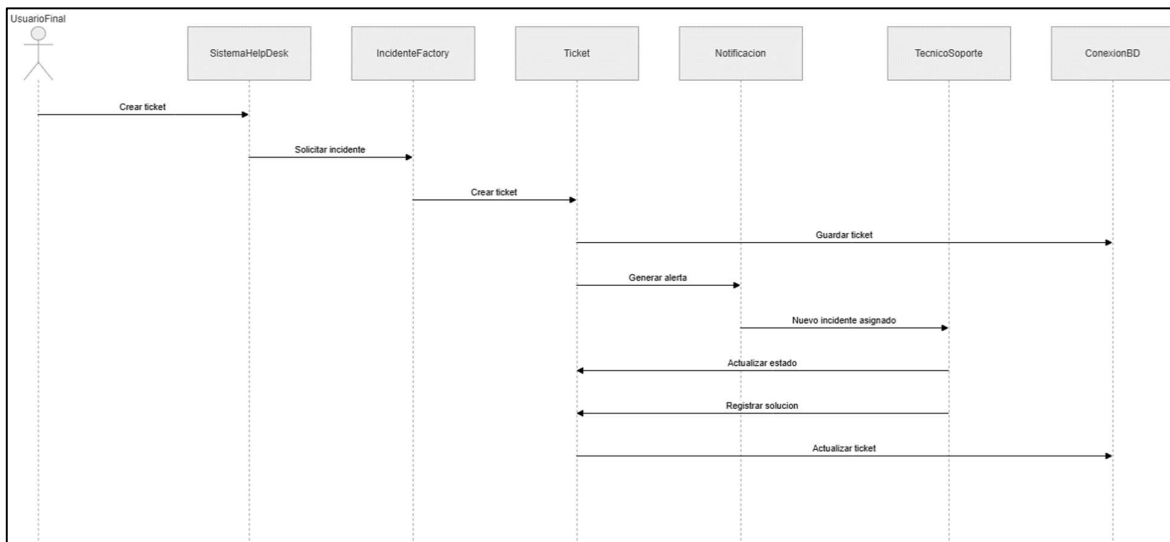
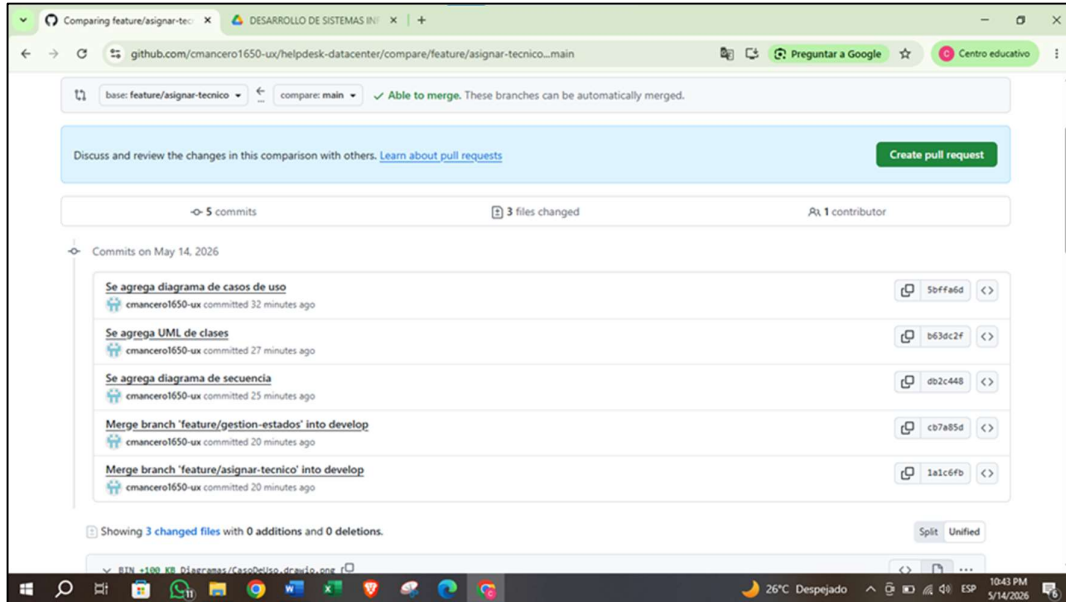
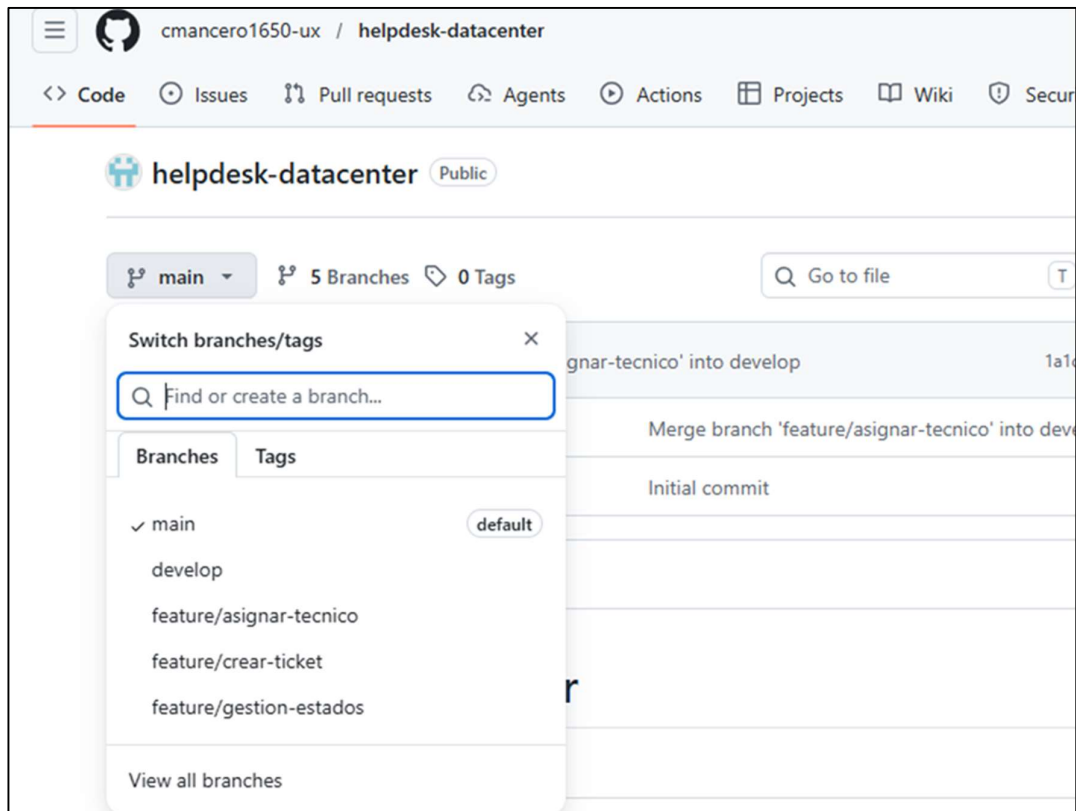


DIAGRAMA DE ARQUITECTURA





IMPLEMENTACIÓN DE GITHUB





Comparing feature/crear-ticket... x DESARROLLO DE SISTEMAS IN... x +

github.com/cmancero1650-ux/helpdesk-datacenter/compare/feature/crear-ticket...main

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base: feature/crear-ticket ← compare: main ✓ Able to merge. These branches can be automatically merged.

Discuss and review the changes in this comparison with others. [Learn about pull requests](#) [Create pull request](#)

4 commits 2 files changed 1 contributor

Commits on May 14, 2026

- Se agrega UML de clases
cmancero1650-ux committed 27 minutes ago b63dc2f <>
- Se agrega arquitectura del sistema
cmancero1650-ux committed 25 minutes ago 71697ea <>
- Merge branch 'feature/gestion-estados' into develop
cmancero1650-ux committed 21 minutes ago cb7a85d <>
- Merge branch 'feature/asignar-tecnico' into develop
cmancero1650-ux committed 21 minutes ago 1a1c6fb <>

Showing 2 changed files with 0 additions and 0 deletions. Split Unified

Windows taskbar: 26°C Despejado 10:43 PM 5/14/2026

github.com/cmancero1650-ux/helpdesk-datacenter

cmancero1650-ux / helpdesk-datacenter

feature/gestion-estados had recent pushes 3 minutes ago [Compare & pull request](#)

feature/crear-ticket had recent pushes 1 minute ago [Compare & pull request](#)

feature/asignar-tecnico had recent pushes 24 seconds ago [Compare & pull request](#)

main 5 Branches 0 Tags Go to file Add file Code

cmancero1650-ux Initial commit 5bab1a0 - 2 weeks ago 1 Commit

README.md Initial commit 2 weeks ago

README

helodesk-datacenter

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package



```
MINGW64/c/Users/Papeleria/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter
Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/gestion-estados)
$ git branch
  develop
  feature/assignar-tecnico
  feature/crear-ticket
* feature/gestion-estados
  main
  master

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/gestion-estados)
$ git checkout feature/crear-ticket
Switched to branch 'feature/crear-ticket'

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/crear-ticket)
$ git add .

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/crear-ticket)
$ git commit -m "Se agrega diagrama de casos de uso"
[feature/crear-ticket 5bffa6d] Se agrega diagrama de casos de uso
1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Diagramas/CasoDeUso.drawio.png

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/crear-ticket)
$ git push origin feature/crear-ticket
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 82.22 KiB | 16.44 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/cmancero1650-ux/helpdesk-datacenter.git
 5babiad..5bffa6d feature/crear-ticket -> feature/crear-ticket

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/crear-ticket)
$ git checkout feature/gestion-estados
Switched to branch 'feature/gestion-estados'

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/gestion-estados)
$ git checkout feature/crear-ticket
Switched to branch 'feature/crear-ticket'

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/crear-ticket)
$ git checkout feature/gestion-estados
Switched to branch 'feature/gestion-estados'

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/gestion-estados)
$ git add .

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/gestion-estados)
```

```
MINGW64/c/Users/Papeleria/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter
Switched to branch 'feature/gestion-estados'

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/gestion-estados)
$ git add .

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/gestion-estados)
$ git commit -m "Se agrega UML de clases"
[feature/gestion-estados b63dc2f] Se agrega UML de clases
1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Diagramas/ClaseUML.drawio.png

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/gestion-estados)
$ git push origin feature/gestion-estados
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 116.55 KiB | 14.57 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/cmancero1650-ux/helpdesk-datacenter.git
 5babiad..b63dc2f feature/gestion-estados -> feature/gestion-estados

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/gestion-estados)
$ git checkout feature/crear-ticket
Switched to branch 'feature/crear-ticket'

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/crear-ticket)
$ git add .

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/crear-ticket)
$ git commit -m "Se agrega diagrama de secuencia"
[feature/crear-ticket db2c448] Se agrega diagrama de secuencia
1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Diagramas/Secuencia.drawio.png

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/crear-ticket)
$ git push origin feature/crear-ticket
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 45.73 KiB | 14.58 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/cmancero1650-ux/helpdesk-datacenter.git
 5bffa6d..db2c448 feature/crear-ticket -> feature/crear-ticket

Papelela@DESKTOP-8C37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/crear-ticket)
$ git checkout feature/assignar-tecnico
Switched to branch 'feature/assignar-tecnico'
```



```
MINGW64~/Users/Papeleira/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter

Papeleria@DESKTOP-BC37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/gestion-estados)
$ git checkout feature/crear-ticket
Switched to branch 'feature/crear-ticket'

Papeleria@DESKTOP-BC37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/crear-ticket)
$ git add .

Papeleria@DESKTOP-BC37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/crear-ticket)
$ git commit -m "Se agrega diagrama de secuencia"
[feature/crear-ticket db2c448] Se agrega diagrama de secuencia
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Diagramas/Secuencia.drawio.png

Papeleria@DESKTOP-BC37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/crear-ticket)
$ git push origin feature/crear-ticket
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 43.73 KiB | 14.58 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/cmacero1650-ux/helpdesk-datacenter.git
  bbfad..db2c448 feature/crear-ticket -> feature/crear-ticket

Papeleria@DESKTOP-BC37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/crear-ticket)
$ git checkout feature/asignar-tecnico
Switched to branch 'feature/asignar-tecnico'

Papeleria@DESKTOP-BC37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/asignar-tecnico)
$ git add .

Papeleria@DESKTOP-BC37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/asignar-tecnico)
$ git commit -m "Se agrega arquitectura del sistema"
[feature/asignar-tecnico 71697ea] Se agrega arquitectura del sistema
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Diagramas/Arquitectura.drawio.png

Papeleria@DESKTOP-BC37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (feature/asignar-tecnico)
$ git push origin feature/asignar-tecnico
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 58.76 KiB | 14.69 MiB/s, done.
```

```
MINGW64~/Users/Papeleira/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter

Merge made by the 'ort' strategy.
Diagramas/Arquitectura.drawio.png | Bin 0 -> 83178 bytes
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Diagramas/Arquitectura.drawio.png

Papeleria@DESKTOP-BC37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (develop)
$ git merge feature/asignar-tecnico
Already up to date.

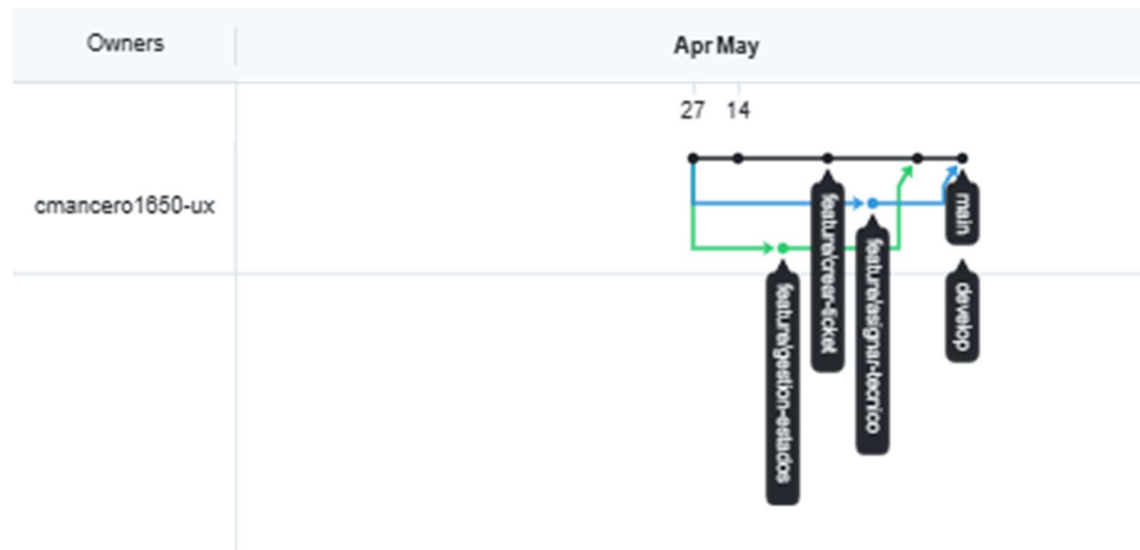
Papeleria@DESKTOP-BC37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (develop)
$ git push origin develop
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 799 bytes | 99.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/cmacero1650-ux/helpdesk-datacenter.git
  5bba1a0..1a1c6fb develop -> develop

Papeleria@DESKTOP-BC37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (develop)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

Papeleria@DESKTOP-BC37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (main)
$ git merge develop
Updating 5bba1a0..1a1c6fb
Fast-forward
Diagramas/Arquitectura.drawio.png | Bin 0 -> 83178 bytes
Diagramas/CasoDeUso.drawio.png | Bin 0 -> 103400 bytes
Diagramas/ClassML.drawio.png | Bin 0 -> 179244 bytes
Diagramas/Secuencia.drawio.png | Bin 0 -> 67239 bytes
4 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Diagramas/Arquitectura.drawio.png
create mode 100644 Diagramas/CasoDeUso.drawio.png
create mode 100644 Diagramas/ClassML.drawio.png
create mode 100644 Diagramas/Secuencia.drawio.png

Papeleria@DESKTOP-BC37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (main)
$ git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/cmacero1650-ux/helpdesk-datacenter.git
  5bba1a0..1a1c6fb main -> main

Papeleria@DESKTOP-BC37D72 MINGW64 ~/Desktop/STO SEMESTRE/DESARROLLO DE SISTEMAS INFORMATICOS/TAREA 1 - HELP DESK/helpdesk-datacenter (main)
$
```



LINK :

<https://github.com/cmancero1650-ux/helpdesk-datacenter.git>